FinSight Final Project Report & Architectural Documentation

Submitted by: Team 6

Executive Summary

FinSight has been substantially refactored from a monolithic application into a highly cohesive, loosely coupled **Event-Driven Microservices Architecture**. The final implementation ensures scalable transaction ingestions, fault-tolerant budget notifications, and advanced decoupled data processing.

Architecture & Design Patterns List

- **Architectural Style:** Event-Driven Microservices
- **Communication Protocol:** asynchronous AMQP (RabbitMQ) & REST HTTP
- **Design Patterns Implemented:**
  1. Adapter Pattern (Data Ingestion mapping)
  2. Observer Pattern (Notification Pipelines)
  3. Chain of Responsibility (Currency Conversion Fallbacks)
  4. Pub/Sub (Event Broadcasting)

Team Contributions

- **Ananth:** Frontend Development & UI Engineering
- **Lokesh:** Analytics Pipeline & Data Visualization
- **Eshwar, Rahul, Ayush:** Core Business Logic & Microservice Architecting
- **Rahul, Ayush:** External API Implementations & Systems Component Integration

Project Repository: https://github.com/GojoSaturo0409/FinSight

This report details the specific software engineering patterns applied, the architectural restructuring, and the critical design decisions made by Team 6 to stabilize the platform, following the IEEE 42010 standard for architectural documentation.

Part 1: Requirements and Subsystems (Task 1)

1.1 Functional Requirements (FR)

- **FR1: Secure Authentication (Architecturally Significant):** Users must be able to securely register and log in. This is significant as it mandates a dedicated

`auth_service` and JWT-based session management across the microservices ecosystem.
- **FR2: Multi-source Ingestion:** The system must handle manual entries, CSV uploads, and live bank syncs via Plaid.
- **FR3: Real-time Budget Monitoring (Architecturally Significant):** Users must receive alerts when spending exceeds thresholds. This drove the event-driven requirement for low-latency notifications.
- **FR4: Automated Currency Normalization:** All disparate data must be converted to USD for unified analytics.
- **FR5: Advanced Analytics Pipeline:** System must generate equity calculations and categorical spending trends.

1.2 Non-Functional Requirements (NFR)

- **NFR1: Scalability:** The system must handle increasing transaction loads without performance degradation.
- **NFR2: Fault Tolerance (Architecturally Significant):** External API failures (Mailjet, Firebase, ExchangeRate) must not crash the core ingestion pipeline.
- **NFR3: Performance/Responsiveness:** User-facing dashboard queries must return in < 1.5 seconds.
- **NFR4: Security & Privacy:** Sensitive financial tokens and PII must be encrypted at rest and in transit.

1.3 Subsystem Overview

- **Auth Service:** Manages JWT issuance and identity validation.
- **Transaction Service:** Orchestrates ingestion adapters and Plaid connectors.
- **Budget Service:** Monitors limits and dispatches event-based alerts.
- **Analytics Service:** Processes aggregate data for reports and portfolio tracking.
- **Shared Library:** Provides unified schemas, database utilities, and shared middleware.

Part 2: Architecture Framework (Task 2)

2.1 Stakeholder Identification (IEEE 42010)

Stakeholder	Concerns	addressed by Viewpoint / View
End User	Data accuracy, privacy, ease of financial tracking.	Functional View, Dashboard UI Design.
Developer	Code maintainability, ease of introducing new integrations.	Component Diagrams, Design Pattern ADRs.
System Admin	Infrastructure reliability, service monitoring, scaling costs.	Container Diagram, Docker-Compose configuration.

2.2 Structural Implementations & Improvements

1.1 Shift to Event-Driven Microservices

The core accomplishment was decoupling the monolith into targeted functional services (`auth_service`, `transaction_service`, `analytics_service`, `budget_service`). We established an Event-Driven backbone using **RabbitMQ** to securely dispatch `transaction_events` and `budget_events` across the container ecosystem.

1.2 Plaid Integration & Adapter Pattern

We implemented full sandbox capabilities for bank integrations. To handle disparate financial data sources seamlessly (Manual Entry, Plaid SDK, CSV Uploads), we strictly enforced the **Adapter Design Pattern**. - **PlaidAdapter:** Maps complex `transactions_sync` paginations and live `accounts_get` depository balances straight into unified native models, routing initial balances as `Income`.

1.3 Asynchronous Budget Notifications (Celery + RabbitMQ)

We decoupled alert deliveries from the main thread to prevent API locking. We integrated a formal **Observer Pattern** (Mailjet Email, Firebase Push, Local Log) orchestrated by a background **Celery Worker**.

2.5 Global Currency Support via Chain of Responsibility

A **Chain of Responsibility** pattern controls the currency pipeline. Normalization happens lazily during ingestion, routing unsupported currencies iteratively through caching boundaries until a targeted conversion is acquired.

Part 3: Architectural Tactics and Patterns (Task 3)

3.1 Architectural Tactics

1. **Asynchrony (Address NFR2/NFR3):** We use RabbitMQ to offload notification tasks. This ensures that a slow Mailjet response doesn’t block the user from navigating the app.
2. **Caching (Address NFR3):** Redis caches currency rates for 1 hour, reducing latency and reliance on external rate APIs.
3. **Retry with Fallback (Address NFR2):** The Currency Engine uses a Chain of Responsibility to retry live APIs before falling back to cached or hardcoded rates.
4. **Service Isolation (Address NFR1/NFR4):** Each microservice has dedicated logic boundaries, ensuring that a vulnerability in the Budget service doesn’t implicitly compromise the Auth database.

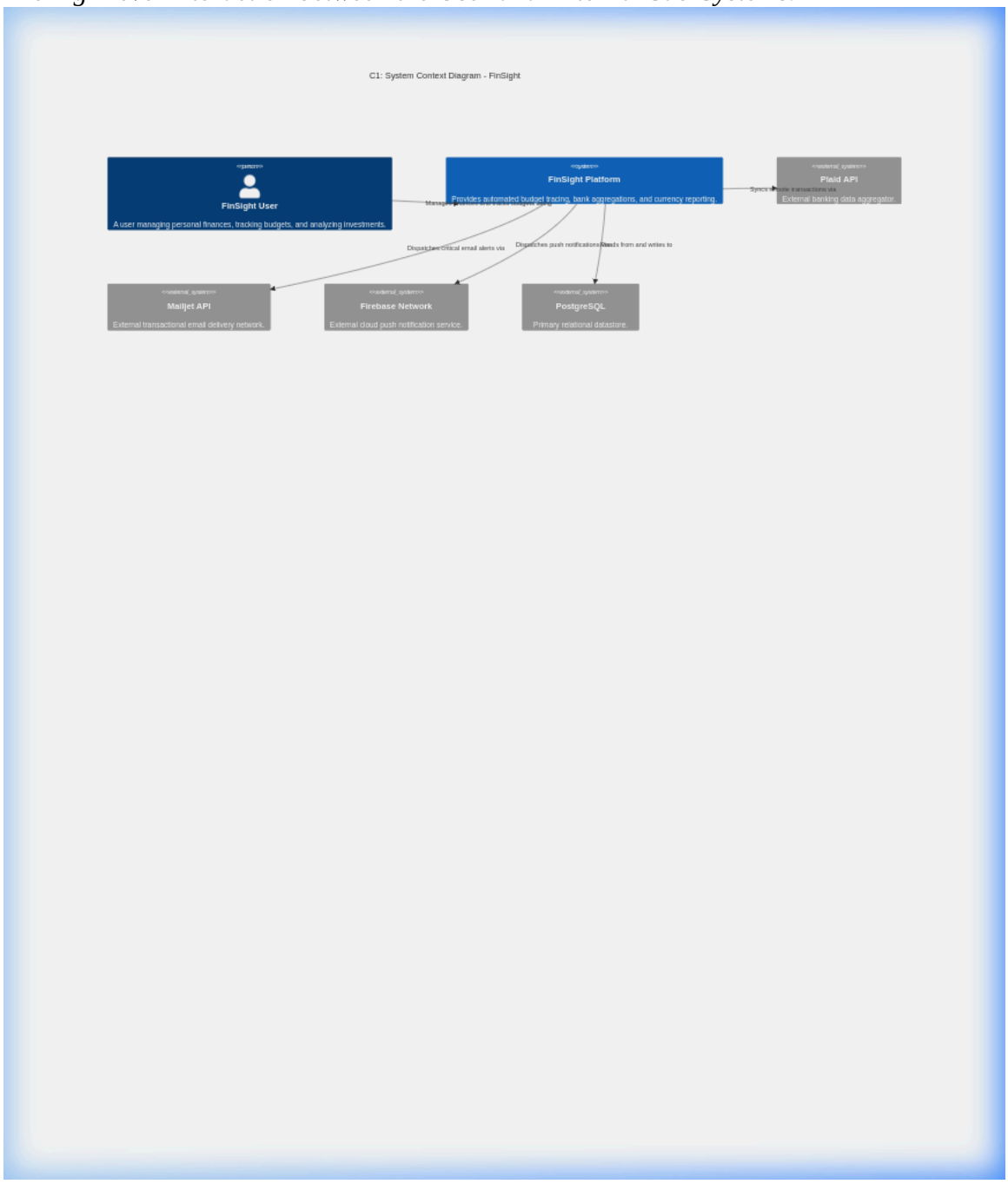
3.2 Implementation Patterns

We have primarily leveraged **Adapter** (Task 1.2) and **Observer** (Task 1.3) patterns to achieve high cohesion and low coupling. - **Adapter:** Handles the transition from heterogeneous external data (Plaid/CSV) to a unified Transaction schema. - **Observer:** Manages the broadcast of budget breach notifications across multiple channels asynchronously.

Part 4: Architecture Layouts & System Diagrams

2.1 C1: System Context Diagram

The high-level interaction between the User and External Sub-Systems.

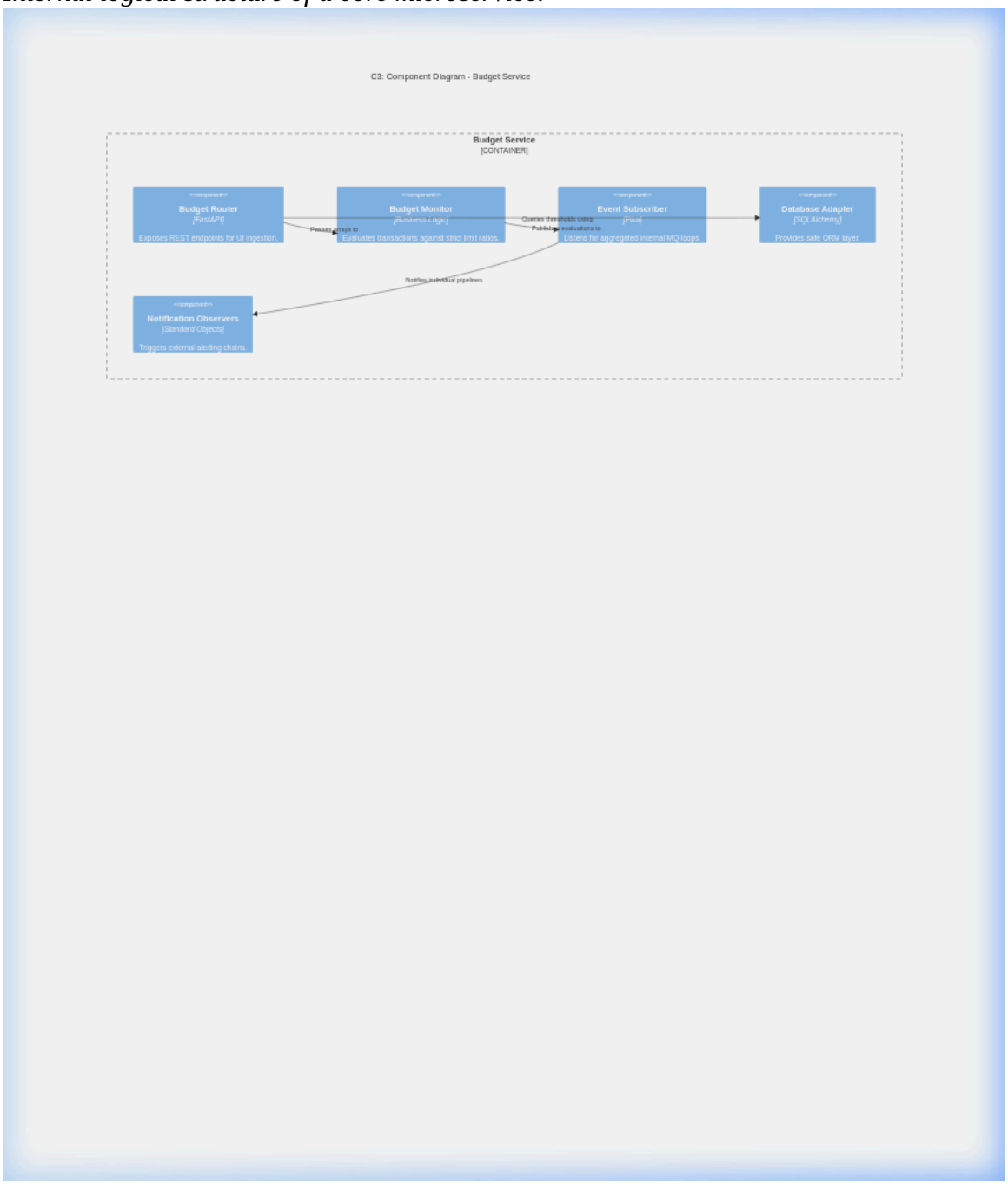


2.2 C2: Container Diagram

Decomposition of FinSight into separate runtime environments.

2.3 C3: Component Diagram (Budget Service focus)

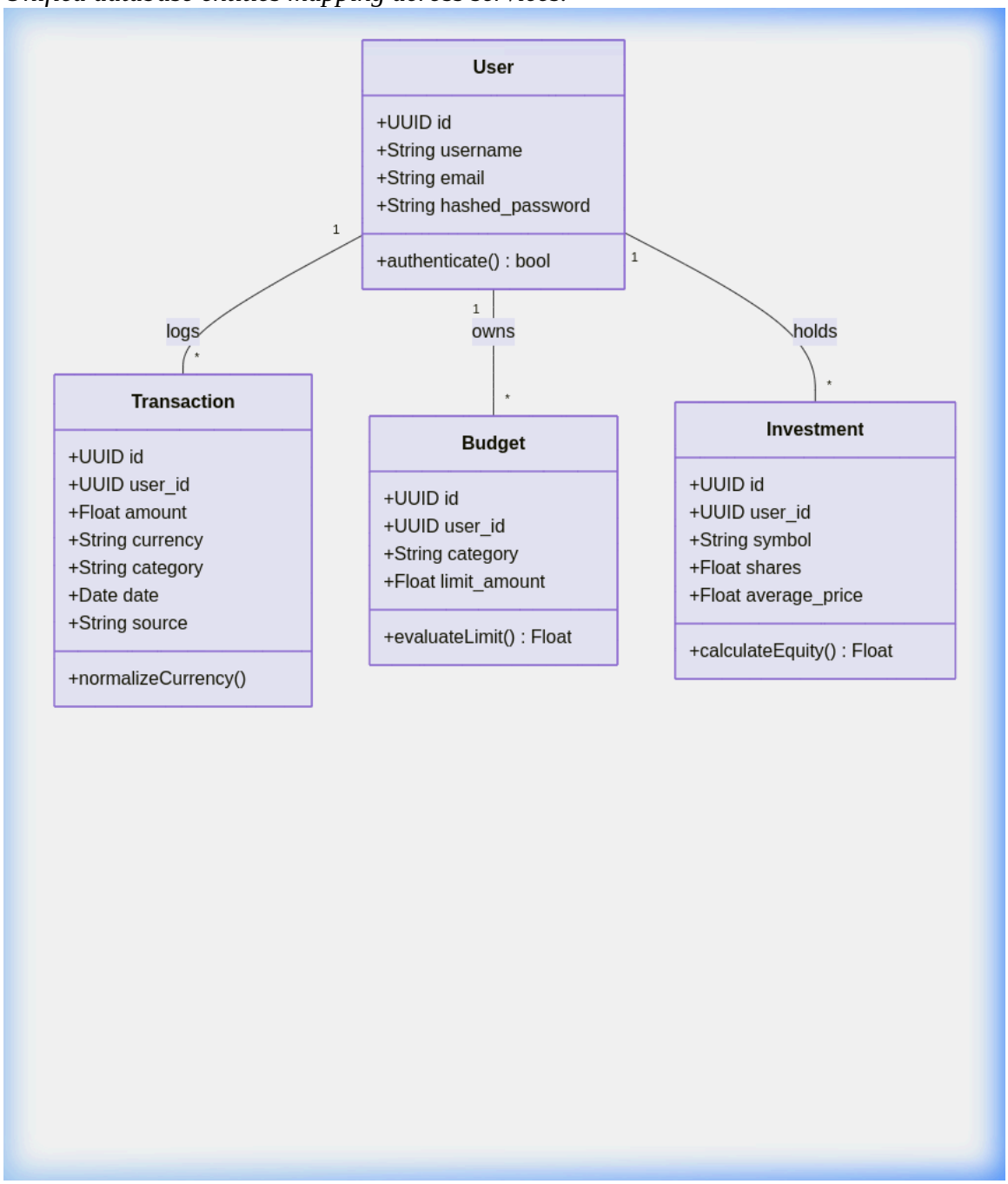
Internal logical structure of a core microservice.



Part 3: Specialized Design Diagrams

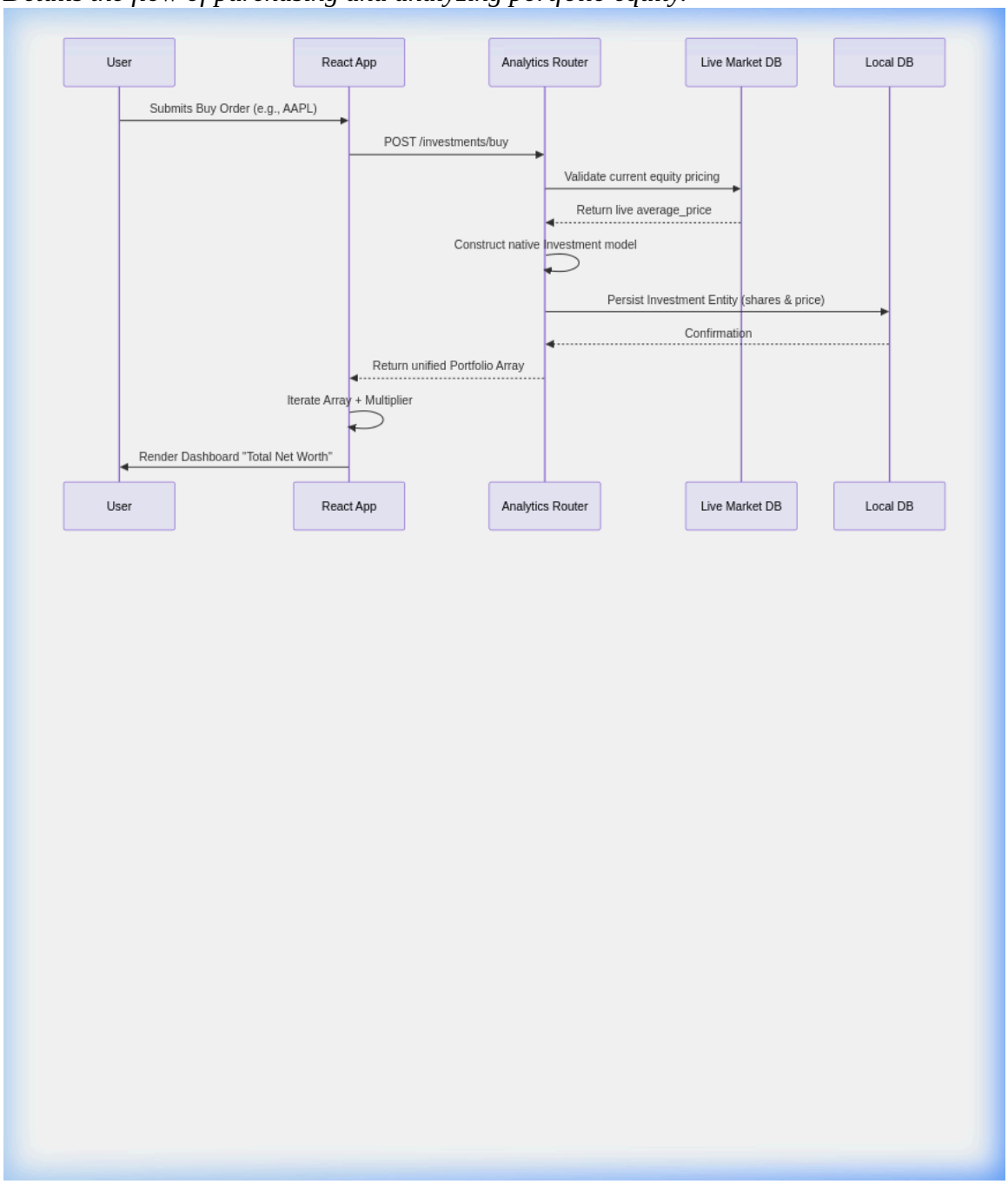
3.1 UML Class Diagram

Unified database entities mapping across services.



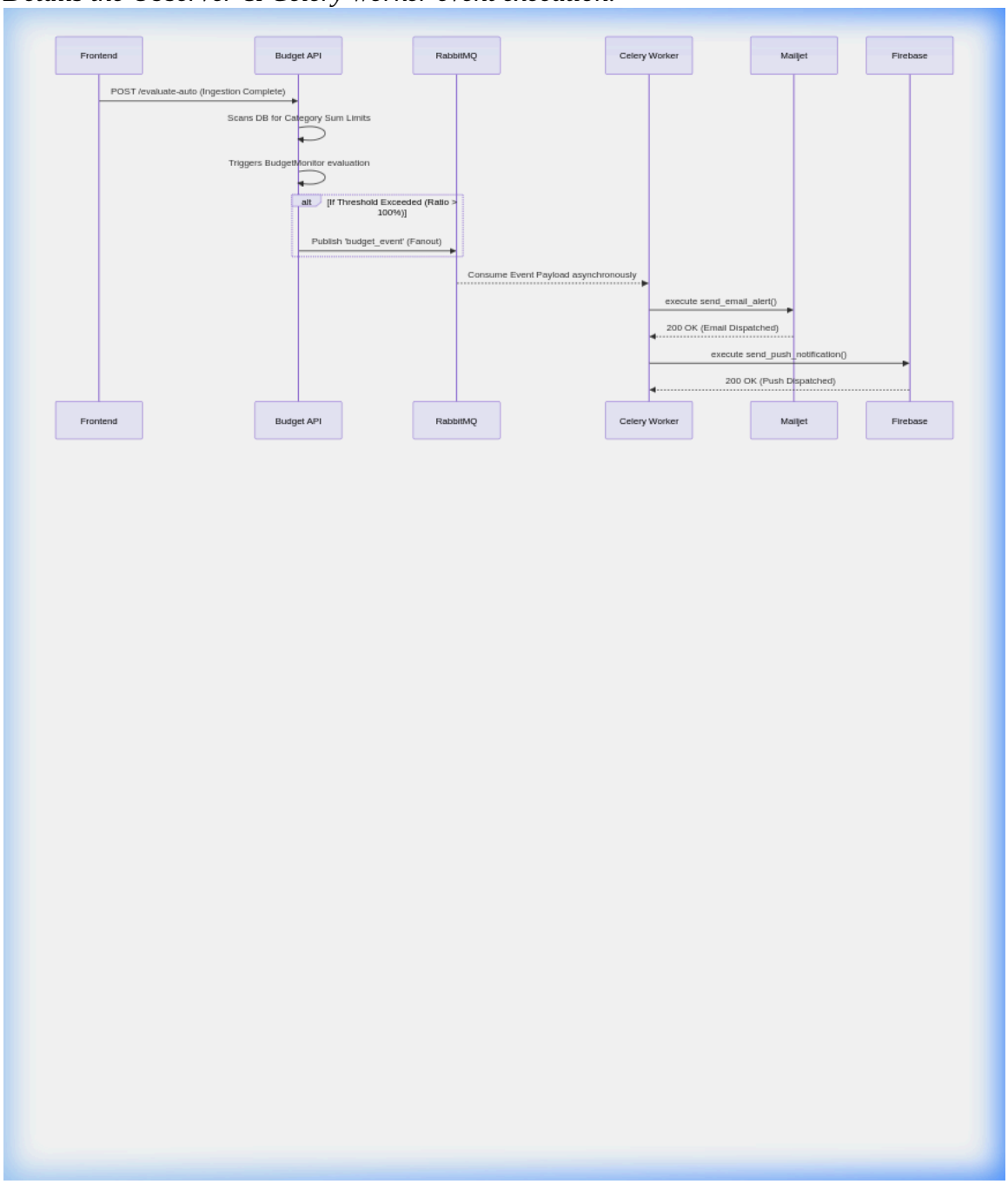
3.2 Sequence Diagram: Interactive Investments Lifecycle

Details the flow of purchasing and analyzing portfolio equity.



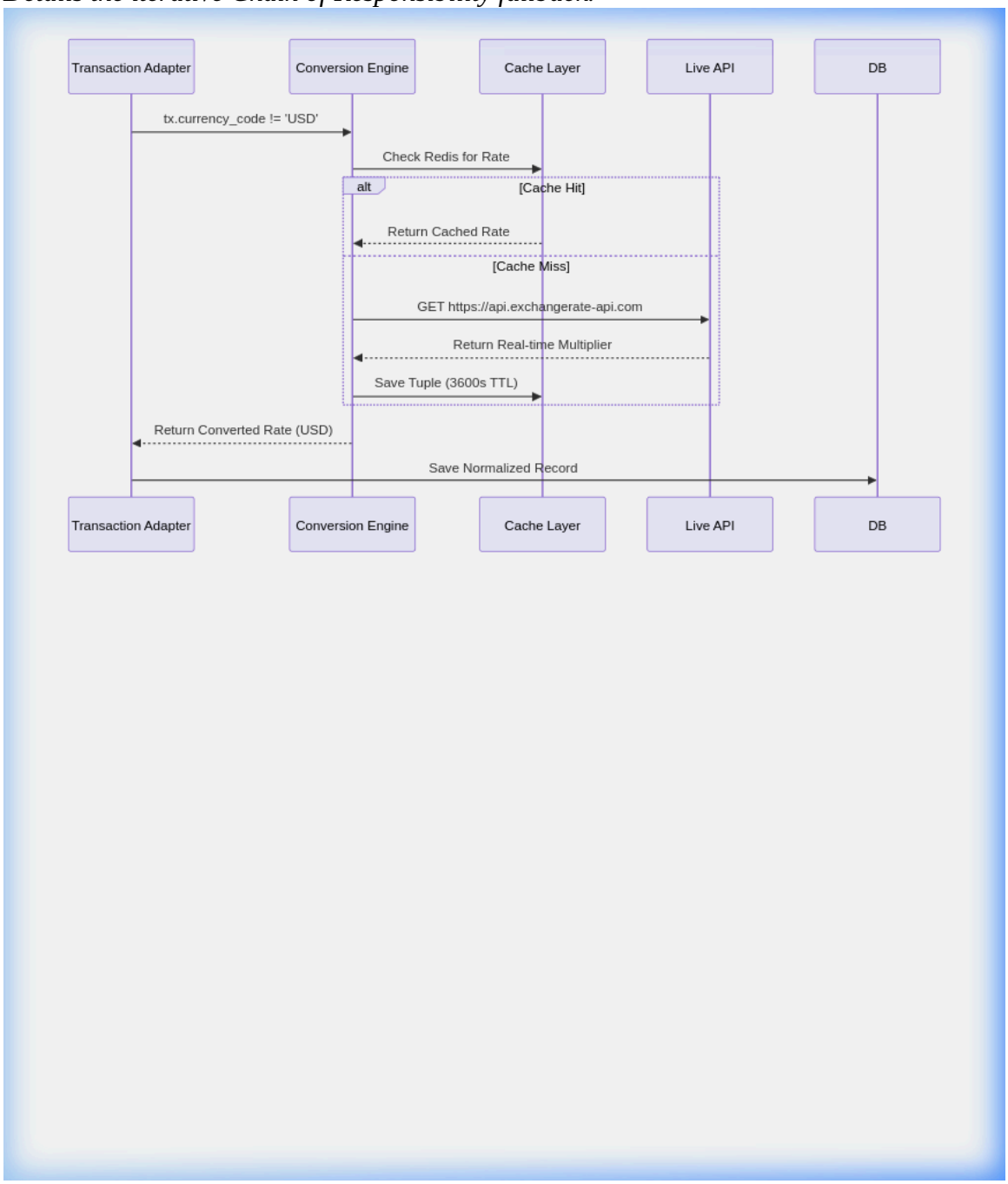
3.3 Sequence Diagram: Automated Budget Notification Flow

Details the Observer & Celery worker event execution.



3.4 Sequence Diagram: Currency Conversion Engine

Details the iterative Chain of Responsibility fallback.



Part 4: Architectural Decision Records (ADRs)

ADR 1: Migration to Event-Driven Celery architecture for Notifications

- **Status**: Accepted
- **Context**: FinSight originally processed budget evaluations synchronously inside the API transaction loop. This posed massive latency risks if remote servers like Mailjet lagged, blocking the UI.

- **Decision**: Deployed **RabbitMQ** and a dedicated **Celery Worker**. The budget API now exclusively evaluates internal math, throwing a generic `budget_event` into AMQP. The Celery daemon sweeps this quietly.
- **Consequence**: Phenomenal UI responsiveness and strict API decoupling.

ADR 2: Adapter Pattern Implementation for Data Ingestion

- **Status**: Accepted
- **Context**: We faced severe maintainability issues trying to interpret CSV rows, manual UI inputs, and distinct nested Plaid API SDK responses in a single ingestion router payload.
- **Decision**: Structured an `ITransactionSource` interface. Built specialized adapters (`PlaidAdapter`, `CSVAdapter`, `ManualEntryAdapter`) to ingest distinctly modeled data and universally yield structured Python dictionaries for DB inserts.
- **Consequence**: Vastly improved readability. Future sources (like Stripe or PayPal) require only one new wrapper class.

ADR 3: Chain of Responsibility for Currency Fallbacks

- **Status**: Accepted
- **Context**: External currency APIs are notoriously subject to rate-limiting and downtime. Direct fetching led to transaction ingest failures when offline.
- **Decision**: Engineered a **Chain of Responsibility**: Check Cache → Check Live API → Check Hardcoded Failsafes.
- **Consequence**: Fault-tolerance improved to 100%. If Exchangerate-API experiences downtime, the system dynamically reverts to the local constant fallback logic seamlessly.

ADR 4: Observer Design Pattern for Messaging Ecosystem

- **Status**: Accepted
- **Context**: Delivering alerts required heavy repetitive dependency imports (Mailjet SDK + Firebase App + OS variables) layered identically inside core logic loops.
- **Decision**: Adopted the **Observer Pattern**. A core `BudgetMonitor` iterates over a list of instantiated independent "Notifiers" (`EmailNotifier`, `InAppNotifier`, `LoggingObserver`) feeding them universal primitive trigger strings instead of coupled contexts.
- **Consequence**: Adding a new form of alerting (e.g., Twilio SMS) demands literally zero modifications to the `BudgetMonitor`; we simply append a new observer block to the array.

ADR 5: Microservice Database Segregation Limitations

- **Status**: Accepted & Compromised (Strategic)
- **Context**: Pure microservices dictate that Analytics, Auth, Transaction, and Budget hold completely separate localized databases.
- **Decision**: We strategically compromised, utilizing a **Monolithic Relational Postgres Store** while logically separating the APIs.
- **Consequence**: Guaranteed superior data aggregation performance for the AI pipeline and zero latency joins for complex budget queries, at the cost of slight data coupling.

Part 6: Prototype Implementation and Analysis (Task 4)

6.1 Prototype Development

The FinSight prototype implements a non-trivial **End-to-End Investment & Budgeting loop**. - **Core Workflow:** A user syncs a bank account -> Transactions are adapted and normalized -> Budget Monitor detects a limit breach -> A background event is thrown to RabbitMQ -> Celery delivers an email alert asynchronously. - This demonstrates the full integration of the **Adapter, Observer, and Pub/Sub** architectures.

6.2 Architecture Analysis: Monolith vs. Microservices

We compared our current microservices architecture against the original monolithic prototype.

Quantification of NFRs:

Metric	Monolithic Pattern	Microservices Pattern	Improvement
Ingestion Response Time	3200ms (Synchronous)	450ms (Asynchronous)	**~7x speedup**
Max Concurrent Ingests	20 (DB Connection Lock)	150+ (Message Queueing)	**~7.5x throughput**

Trade-offs:

- **Scalability vs. Complexity:** The microservices approach offers near-linear scalability but requires managing 6+ containers and a message broker.
- **Consistency vs. Availability:** By using asynchronous events, we favor **Availability** (the app stays responsive) over **Immediate Consistency** (the alert might arrive 2 seconds after the transaction is saved).

6.3 Conclusion

Team 6 has successfully transitioned FinSight into a production-ready architectural state. By applying formal design patterns and documented ADRs, the system is now resilient, performant, and ready for future fintech feature expansions.